PE文件（七）导入表

原创 已于 2025-10-13 11:42:23 修改 · 2k 阅读 · 15 · 16 · CC 4.0 BY-SA版权

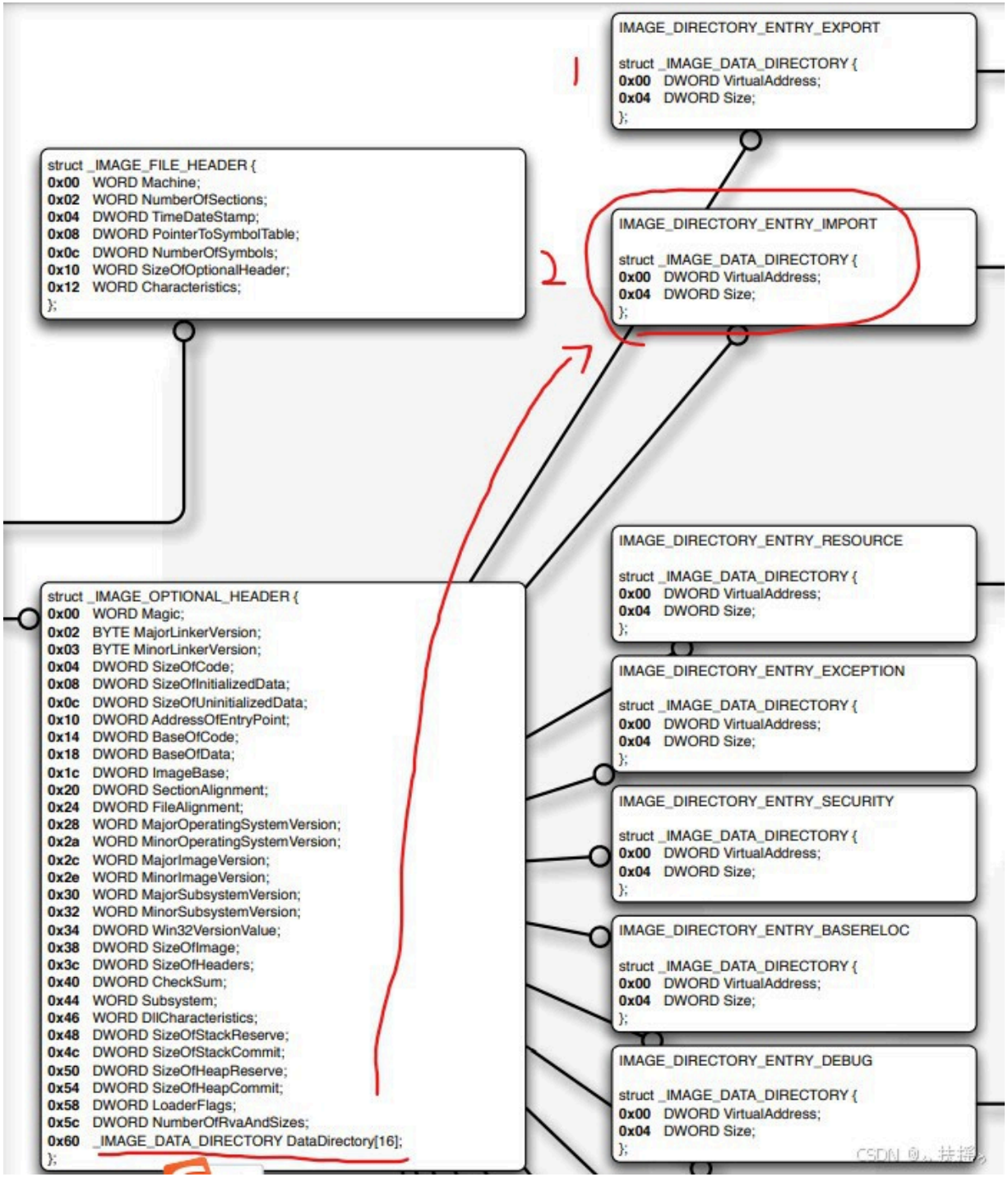
文章标签： #pe文件

导入表的引入

当一个PE文件（如.dll/.exe）需要调用其他模块（默认理解为.dll）的函数时，就需要一份清单记录相关信息，如依赖了哪些dll，使用了dll的哪些函数等等，而这份清单就是导入表。一个dll对应一个导入表，通常一个pe文件不止依赖一个dll，因此一个pe文件通常有多个导入表。

定位导入表

一个PE文件的数据目录第二个结构体就是导入表数据目录，根据导入表数据目录的VirtualAddress成员经过RVA转FOA找到导出表


IMAGE_DIRECTORY_ENTRY_EXPORT
struct _IMAGE_DATA_DIRECTORY {
0x00 DWORD VirtualAddress;
0x04 DWORD Size;
};
1
struct _IMAGE_FILE_HEADER {
0x00 WORD Machine;
0x02 WORD NumberOfSections;
0x04 DWORD TimeDateStamp;
0x08 DWORD PointerToSymbolTable;
0x0c DWORD NumberOfSymbols;
0x10 WORD SizeOfOptionalHeader;
0x12 WORD Characteristics;
};
IMAGE_DIRECTORY_ENTRY_IMPORT
struct _IMAGE_DATA_DIRECTORY {
0x00 DWORD VirtualAddress;
0x04 DWORD Size;
};
2
IMAGE_DIRECTORY_ENTRY_RESOURCE
struct _IMAGE_DATA_DIRECTORY {
0x00 DWORD VirtualAddress;
0x04 DWORD Size;
};
struct _IMAGE_OPTIONAL_HEADER {
0x00 WORD Magic;
0x02 BYTE MajorLinkerVersion;
0x03 BYTE MinorLinkerVersion;
0x04 DWORD SizeOfCode;
0x08 DWORD SizeOfInitializedData;
0x0c DWORD SizeOfUninitializedData;
0x10 DWORD AddressOfEntryPoint;
0x14 DWORD BaseOfCode;
0x18 DWORD BaseOfData;
0x1c DWORD ImageBase;
0x20 DWORD SectionAlignment;
0x24 DWORD FileAlignment;
0x28 WORD MajorOperatingSystemVersion;
0x2a WORD MinorOperatingSystemVersion;
0x2c WORD MajorImageVersion;
0x2e WORD MinorImageVersion;
0x30 WORD MajorSubsystemVersion;
0x32 WORD MinorSubsystemVersion;
0x34 DWORD Win32VersionValue;
0x38 DWORD SizeOfImage;
0x3c DWORD SizeOfHeaders;
0x40 DWORD CheckSum;
0x44 WORD Subsystem;
0x46 WORD DllCharacteristics;
0x48 DWORD SizeOfStackReserve;
0x4c DWORD SizeOfStackCommit;
0x50 DWORD SizeOfHeapReserve;
0x54 DWORD SizeOfHeapCommit;
0x58 DWORD LoaderFlags;
0x5c DWORD NumberOfRvaAndSizes;
0x60 _IMAGE_DATA_DIRECTORY DataDirectory[16];
};
IMAGE_DIRECTORY_ENTRY_EXCEPTION
struct _IMAGE_DATA_DIRECTORY {
0x00 DWORD VirtualAddress;
0x04 DWORD Size;
};
IMAGE_DIRECTORY_ENTRY_SECURITY
struct _IMAGE_DATA_DIRECTORY {
0x00 DWORD VirtualAddress;
0x04 DWORD Size;
};
IMAGE_DIRECTORY_ENTRY_BASERELOC
struct _IMAGE_DATA_DIRECTORY {
0x00 DWORD VirtualAddress;
0x04 DWORD Size;
};
IMAGE_DIRECTORY_ENTRY_DEBUG
struct _IMAGE_DATA_DIRECTORY {
0x00 DWORD VirtualAddress;
0x04 DWORD Size;
};

导入表结构

如下是导入表结构，20字节大小：

cpp　AI构建项目 | 复制 ▷ 运行

```cpp
struct _IMAGE_IMPORT_DESCRIPTOR{
    union{
        DWORD Characteristics;//没啥用
        DWORD OriginalFirstThunk;
    };
    DWORD TimeDateStamp;
    DWORD ForwarderChain;
    DWORD Name;
    DWORD FirstThunk;
};
```

收起 ^

DWORD OriginalFirstThunk：该值为指向INT表的RVA

DWORD FirstThunk：该值为导入地址表（即IAT表）的RVA，当OriginalFirstThunk失效时才有用，在文件加载内存前相当于备份

DWORD TimeDateStamp：时间戳，用于判断该导入表结构对应的DLL中函数绝对地址是否已经绑定IAT表。当该值为0xFFFFFFFF时，表示这个导入表结构对应的DLL中函数绝对地址已经绑定到IAT表中当该值为0x00000000时，表示表示这个导入表结构对应的DLL中的函数绝对地址没有绑定到IAT表中。

DWORD Name：导入表对应.dll的名称的RVA。如果Name指向的.dll名称为“user32.dll”，那么这个导入表中记录的就是该PE文件使用user32.dll的相关信息。

注意：IAT表和INT表的结构是一样的

INT表

INT表是一个IMAGE_THUNK_DATA 结构体数组，该结构体是一个四字节的联合体，存储了导入函数相关的信息，结构如下：

cpp　AI构建项目 | 复制 ▷ 运行

```cpp
typedef struct _IMAGE_THUNK_DATA32 {
```

```
    union { 3         DWORD ForwarderString;      //没啥用
        DWORD Function;             //导入函数地址
        DWORD Ordinal;              //导入函数序号
        DWORD AddressOfData;        //_IMAGE_IMPORT_BY_NAME的RVA
    } u1;
} IMAGE_THUNK_DATA32;
```

一个.DLL中的函数以可以函数名称导出，也可以以序号（NONAME）导出。当导出函数没有给出名称，只有序号时，该结构体便以Ordinal的形式存在。

在我们实际应用中，把该结构体直接当成一个DWORD变量即可。当该变量值为最高位为1时，去掉最高位的值就是函数的导出序号。如果最高位不是1，那么该变量值就是指向导入函数名称表的RVA

当PE文件未加载内存时，该结构体的成员为AddressOfData：_IMAGE_IMPORT_BY_NAME结构体的RVA。该结构体如下所示：

cpp　AI构建项目 | 复制　▷ 运行

```cpp
struct _IMAGE_IMPORT_BY_NAME{
    WORD Hint;      //导入函数在其dll导出表中的索引提示，起辅助作用
    BYTE Name[1];   //函数名称，以0结尾
};
```

Hint：这个值由编译器决定是否为空，如果不为空，表示函数在导出表中的索引。一般不使用这个值

Name：由于函数名称的字符个数是不确定的，所以此处只给出一个1字节的字符数组用来存储函数名称的第一个字符。通过Name获取该字符串的首地址，再依次往后遍历，直到遇到字符串的结尾字符\0，便是完整的函数名称

文件加载内存前

由于IAT表和INT表在PE文件加载内存前后会发生变化，因此我们需要分析两种情况

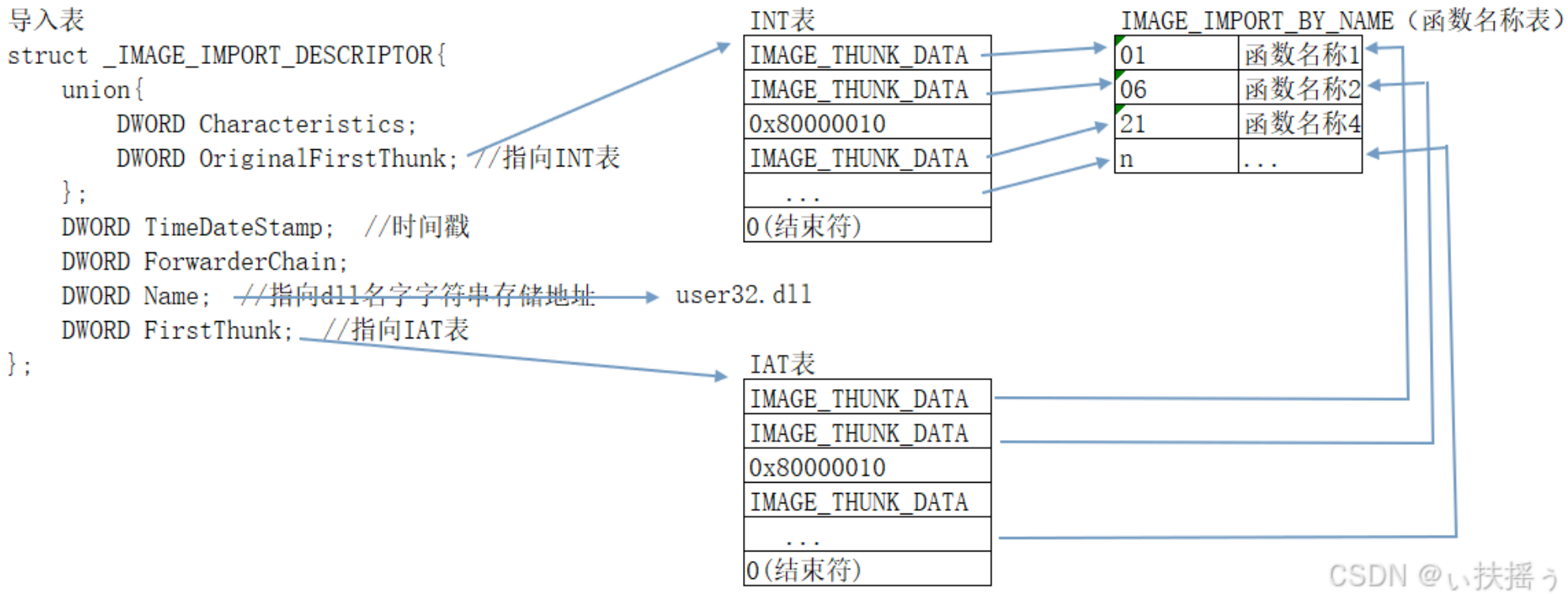
在加载内存前，INT表和IAT表都有各自的内存空间，但它们的结构和内容是完全一样的，都是原始的样子，都叫做原始未绑定的导入地址表。因此INT表虽然叫做导入名称表，但实际是原始未绑定的导入地址表

IAT表有两种定位的方法：

1.通过导入表中的FirstThunk（RVA）成员，找到第一个IAT表

2.通过数据目录表中第13个成员，找到找到第一个IAT表

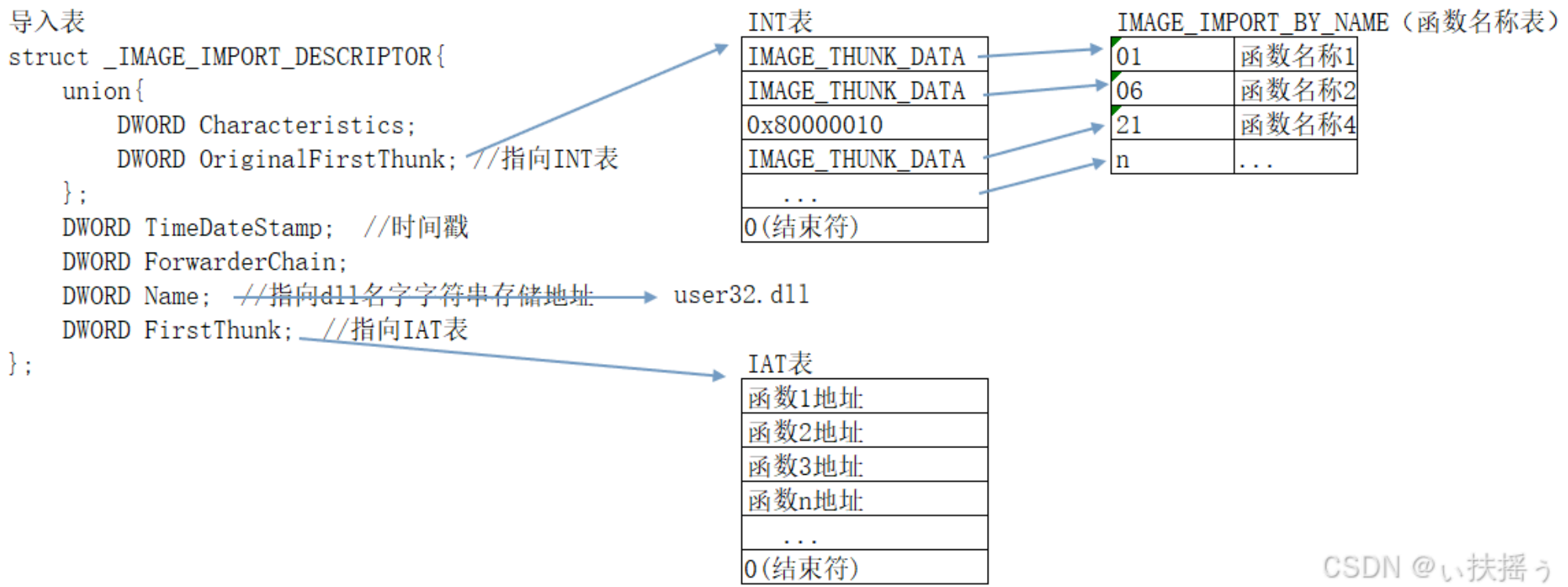
当找到IAT表后往后依次遍历出现有连续四字节长度的0时，表示该IAT表结束



表中有两个内容IMAGE_THUNK_DATA结构体和0x80000010（函数序号）

文件加载内存后

PE文件加载内存后，操作系统调用GetProcAddr函数，循环遍历导出表，通过表中函数名称或者函数序号获取函数地址，然后覆盖到对应的IAT表相关位置。此时IAT表叫做绑定导入地址表，是它原本的样子，表如其名：导入地址表，用于存储导入函数的实际内存地址



总结:

1.当一个导入表后跟了一个导入表大小即20个字节长度的0时，表示该PE文件的所有导入表结束。

2.一般来说，程序通过导入表中的信息来装载对应的.dll文件到虚拟内存中（比如通过导入表的所有Name成员，获取系统要装载的.DLL的名字）。但导入表中的某些信息则需要在.DLL装载内存完成后（比如IAT表）才能修复

3.如果导入表的某个结构中OriginalFirstThunk和FirstThunk的值都为0，这意味着该PE文件没有使用这个DLL中的任何函数。此时操作系统就不会加载这个导入表结构对应的DLL

4.一个PE文件的导入表数量就是该PE文件使用到的.dll文件的数量。

5.PE文件中存在IAT表和INT表两张表的原因：如果只有IAT表，当加载内存后，IAT表的内容就变成函数地址了，也就无法得知函数名称了。为了避免这种情况，多了一个INT表用来备份函数名称。

遍历导入表

遍历内存加载前的导入表

```
void ImportTable(char * szBuffer)
{
    PIMAGE_DOS_HEADER pDos = (PIMAGE_DOS_HEADER)szBuffer;
    PIMAGE_NT_HEADERS pNt = (PIMAGE_NT_HEADERS)(szBuffer + pDos->e_lfanew);
    PIMAGE_OPTIONAL_HEADER pOptionHeader = &pNt->OptionalHeader;
    PIMAGE_DATA_DIRECTORY pImportDir = pOptionHeader->DataDirectory + IMAGE_DIRECTORY_ENTRY_IMPORT;
    PIMAGE_IMPORT_DESCRIPTOR pImport = (PIMAGE_IMPORT_DESCRIPTOR)(RvaToFoa(pImportDir->VirtualAddress, szBuffer) + szBuffer);
    //当导入表不为空时
    while (pImport->Name != NULL)
    {
        //遍历每一个导入表的基本信息
        char * szMoudleName = (char *)(RvaToFoa(pImport->Name, szBuffer) + szBuffer);
        printf("模块名称:%s\r\n", szMoudleName);
        printf("日期时间标志:%08X\r\n", pImport->TimeDateStamp);
        printf("ForwarderChain:%08X\r\n", pImport->ForwarderChain);
        printf("名称Offset:%08X\r\n", pImport->Name);
        printf("FirstThunk:%08X\r\n", pImport->FirstThunk);
        printf("OriginalFirstThunk:%08X\r\n\r\n", pImport->OriginalFirstThunk);
        //获取INT表
        PIMAGE_THUNK_DATA pIAT = (PIMAGE_THUNK_DATA)(RvaToFoa(pImport->OriginalFirstThunk, szBuffer) + szBuffer);
        DWORD dwIndex = 0;
        DWORD dwImportOffset = 0;
        //当INT表不为空时
        while (pINT->u1.Ordinal != 0)
        {
            //pImport->OriginalFirstThunk指向INT表
            //PIMAGE_THUNK_DATA结构占四个字节，每次遍历加4
            //注意此处为地址，而非结构体
            printf("ThunkRVA:%08X\r\n", pImport->OriginalFirstThunk + dwIndex);//内存偏移
            dwImportOffset = RvaToFoa(pImport->OriginalFirstThunk, szBuffer);
            printf("ThunkOffset:%08X\r\n", dwImportOffset + dwIndex);//文件偏移
            dwIndex += 4;
            //当函数以名称导入时
```

```
            if ((pINT->u1.Ordinal & 0x80000000) != 1)
            {
                //RVA 内存中的相对虚拟地址
                //FOA 文件偏移地址
                //szBuffer = base = 0
                //szBuffer + offset = PIMAGE_IMPORT_BY_NAME 在文件中的位置
                PIMAGE_IMPORT_BY_NAME pName = (PIMAGE_IMPORT_BY_NAME)(RvaToFoa(pIAT->u1.AddressOfData, szBuffer) + szBuffer);
                __try
                {
                    printf("API名称:%s\r\n", pName->Name);
                    printf("HINT:%04X\r\n", pName->Hint);
                }
                //当函数以序号导入时
                __except (EXCEPTION_EXECUTE_HANDLER)
                {
                    //序号 = 函数地址- 0x80000000
                    int nOun = pIAT->u1.Function - 0x80000000;
                    char szNameBuffer[MAX_PATH] = { 0 };
                    sprintf(szNameBuffer, "序号: %Xh %dd", nOun, nOun);
                    //函数名称以序号的形式输出
                    printf("API名称:%s\r\n", szNameBuffer);
                    printf("HINT:-\r\n");
                }
                printf("ThunkValue:%08X\r\n\r\n", pINT->u1.Function);//导入函数的地址
            }
            pIAT++;
        }
        pImport++;
    }
}
```

收起 ^